CHAPTER 32

Designing Tool Schemas with JSON Schema

In this chapter

Introduction: The Payments Tool That Guaranteed the Wrong Thing . . . 3
32.1 Core Concepts . . . 4
32.2 The Anatomy of a Tool Definition . . . 5
32.3 The Tool-Use System Prompt . . . 6
32.4 What strict: true Guarantees . . . 7
32.5 The Three Limits Are One Budget . . . 7
32.6 Consolidation, Priced . . . 8
32.7 input_examples . . . 8
32.8 tool_choice, and What Each Option Costs . . . 9
32.9 The Array Is a Cache Prefix . . . 10
32.10 Property Ordering . . . 10
32.11 Visual Analogies . . . 10
32.12 Figures . . . 11
32.13 Examples . . . 12
32.17 Code Walkthrough: Counting What Is Enforced . . . 13
32.18 Mini Project: A Tool-Definition Preflight and a Budget . . . 14
32.19 Hands-on Labs . . . 16
32.20 Lab Solutions . . . 17
32.21 Best Practices . . . 17
32.22 Common Mistakes . . . 18
32.23 Performance Considerations . . . 19
32.24 Security Considerations . . . 19
32.25 Real Production Examples . . . 20
32.26 Certification Notes . . . 20
32.27 Twenty Practice Questions . . . 21
32.28 Answers . . . 22
32.29 Ten Practical Exercises . . . 22
32.30 Five Challenge Labs . . . 23
32.31 Ten Interview Questions . . . 23

32.32 Summary 23
32.33 Glossary 25
32.34 Further Reading 26

Part 6 of 13 · Chapter 32 of 64. Next: Chapter 33 — Tool Descriptions, Naming and Affordances.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Write a tool definition that the API accepts, naming every required and optional field and the rules each one carries.
2. State precisely what `strict: true` guarantees and what it does not, and **measure** the gap on a real schema.
3. Predict which constraints the SDKs silently strip into the description, and design so that nothing safety-critical is among them.
4. Count the three strict-mode limits **across a whole request** rather than per tool, and detect a toolset that will be rejected before you send it.
5. Use `input_examples` correctly, and price them.
6. Choose between `auto`, `any`, `tool` and `none`, and say what each costs in tokens, in cache and in compatibility.
7. Consolidate a sprawling toolset into fewer, more capable tools, and justify the change with numbers rather than taste.
8. Keep the `tools` array byte-stable so that Chapter 30's cache survives the whole conversation.

NOTE

Prerequisites

Chapter 23 for constrained decoding, the supported JSON Schema subset, the three combined limits, the SDKs' silent schema transformations and the rule that `oneOf` is documented neither way and must raise. **Chapter 27** for the body's parameters. **Chapter 28** for `stop_reason` and the response shape. **Chapter 30** for the prefix hierarchy `tools` → `system` → `messages`, and for what a change to `tools` invalidates. **Chapter 31** for the loop, the three execution buckets and the continuation contract.

Deferred by name: **the words in the description**, which are the largest single lever on whether the right tool is called, are **Chapter 33** — this chapter is about the shape. **Error contracts, validation at the call site and idempotency** are **Chapter 34**. **Many tools across many turns** is **Chapter 35**. **Tools that arrive from an MCP server**, and therefore whose schemas you did not write, are **Chapters 36–40**.

Introduction: The Payments Tool That Guaranteed the Wrong Thing

A bank builds a transfer tool. The schema is careful: `amount_minor` is an integer with `minimum: 1` and `maximum: 1000000`, the currency is an `enum`, the idempotency key has a `pattern`, the reference has a `maxLength`. `strict: true` is set, because the documentation is clear that strict mode *"constrains the tool* `input` *and guarantees a valid tool* `name`*"*. The request is accepted, the grammar compiles, and every tool call that comes back parses.

Then a call arrives asking to move `-1` minor units. Another asks for `4000000000`.

Nothing is broken. $32.18 audits that exact schema and finds **sixteen declared constraints, thirteen enforced by the grammar and three stripped into the description** — an enforcement coverage of

81.2 per cent. The three that are not enforced are `minimum`, `maximum` and `maxLength`: the amount's floor, the amount's ceiling and the payment reference's length. Every constraint that protects money is in the stripped set, and every constraint that survives — `type`, `enum`, `pattern`, `required` — is about shape.

This is documented behaviour, not a defect. Chapter 23 recorded it: **all numerical constraints and `minLength`/`maxLength` are outside the supported subset**, and the Python, TypeScript, Ruby and PHP SDKs *"silently transform the schema"* — stripping those keywords into the description, adding `additionalProperties: false`, filtering unsupported formats — and then validating the response client-side against your original. The word doing the damage is **silently**. Your schema type-checks. Your request succeeds. Your grammar compiles. And a fact you wrote down as a constraint has become a sentence of English addressed to a language model.

Chapter 23 gave the running thread: **a grammar constrains the shape of an answer and says nothing about its truth.** This chapter sharpens it for tools, where the answer is a request to do something: a grammar constrains the shape of a *request*, and shape is not authorisation. The design consequence is a rule you can apply mechanically — **never put a safety property somewhere the grammar cannot enforce it** — and the engineering consequence is that validation belongs at the call site, in your code, which is Chapter 34.

There is a second failure in this chapter and it is arithmetical. The three strict-mode limits — 20 strict tools, 24 optional parameters, 16 union-typed parameters — are *"combined across all strict schemas AND the JSON output schema in one request"*. Nine perfectly reasonable single-purpose tools, each with four optional parameters, is **thirty-six optional parameters** and a rejected request, even though no individual tool is anywhere near a limit. $32.18 measures it, and measures what the documented remedy — consolidating related operations behind an `action` parameter — buys back: nine tools and 36 optional parameters become **two tools and 8**.

32.1 Core Concepts

DEFINITION

Tool definition

An entry in the top-level `tools` array. Three required fields — `name`, `description`, `input_schema` — and a set of optional ones including `input_examples`, `cache_control`, `strict`, `defer_loading` and `allowed_callers`.

DEFINITION

`name`

The tool's identifier, which *"must match the regex* `^[a-zA-Z0-9_-]{1,64}$`*"*. No dots, no slashes, no spaces.

DEFINITION

`input_schema`

"A JSON Schema object defining the expected parameters for the tool." Under `strict: true` it is compiled into a grammar; without it, it is documentation the model reads.

DEFINITION

`input_examples`

An optional array of example input objects. *"Each example must be valid according to the tool's `input_schema`. Invalid examples return a 400 error."*

DEFINITION

`strict: true`

Constrained decoding applied to a tool's `input`, plus a guarantee that the emitted tool `name` is one you declared. Subject to the supported subset and the three combined limits.

DEFINITION

The tool-use system prompt

A system prompt the API constructs from your tool definitions, the tool configuration and your own system prompt. It exists whether you asked for it or not, and it costs tokens before any tool is called.

DEFINITION

`tool_choice`

`auto` (default when tools are present), `any`, `tool` or `none` (default when no tools are present). Under `any` and `tool`, *"the API prefills the assistant message to force a tool to be used."*

32.2 The Anatomy of a Tool Definition

Field	Required	Rule
`name`	yes	`^[a-zA-Z0-9_-]{1,64}$`
`description`	yes	*"A detailed plaintext description of what the tool does, when it should be used, and how it behaves."*
`input_schema`	yes	A JSON Schema object
`input_examples`	no	Each must validate against `input_schema`; **not supported on server tools**
`strict`	no	Constrained decoding on the input

Field	Required	Rule
`cache_control`	no	Chapter 30: place it on the **last** tool to cache the whole tool prefix
`defer_loading`	no	Chapter 30: keeps the tool out of the prefix until tool search finds it
`allowed_callers`	no	Which callers may invoke it; taken up in Chapter 35

The names are not merely identifiers. The documentation asks for **meaningful namespacing** — *"prefix names with the service (for example, `github_list_prs`, `slack_send_message`). This makes tool selection unambiguous as your library grows, and is especially important when using tool search."* Sixty-four characters is generous enough that there is no excuse for `get`, and Chapter 33 will show what a name is doing to selection.

32.3 The Tool-Use System Prompt

Sending `tools` changes your prompt whether you meant it to or not. The API constructs a system prompt from the tool definitions, the tool configuration and your own system prompt, in that structure:

```
In this environment you have access to a set of tools you can use to
answer the user's question.
{{ FORMATTING INSTRUCTIONS }}
String and scalar parameters should be specified as is, while lists
and objects should use JSON format. Note that spaces for string values
are not stripped. The output is not expected to be valid XML and is
parsed with regular expressions.
Here are the functions available in JSONSchema format:
{{ TOOL DEFINITIONS IN JSON SCHEMA }}
{{ USER SYSTEM PROMPT }}
{{ TOOL CONFIGURATION }}
```

Two things are worth noticing in a prompt you did not write. Your system prompt sits **between** the tool definitions and the tool configuration — Chapter 21's authority ordering is preserved, but you are not at the top. And the framing sentence, *"The output is not expected to be valid XML and is parsed with regular expressions"*, is a candid statement about the wire format underneath the typed blocks, and it is why Chapter 22's warning about content that speaks your markup applies to tool inputs too.

The overhead is published, and Chapter 11 verified it:

Model	`tool_choice` `auto` / `none`	`any` / `tool`
Claude Opus 5	286	406
Claude Sonnet 5	354	474
Claude Haiku 4.5	496	588

Forcing a tool costs 120 extra tokens on Opus 5 and Sonnet 5, and 92 on Haiku 4.5, on every request. Note the ordering too: Haiku 4.5's tool preamble is the largest of the three, which is the opposite of the intuition that a smaller model has a smaller prompt.

32.4 What `strict: true` Guarantees

Chapter 23 established the mechanism and it has not changed: constrained decoding is *"a restriction on the token distribution at sampling time, not a validator"*. Applied to a tool, `strict: true` guarantees two things — the emitted `name` is one you declared, and the `input` conforms to the **supported subset** of your schema.

That second phrase is the whole chapter. The supported subset, verified in Chapter 23 and unchanged:

Supported: all basic types; `enum` over scalars only; `const`; `anyOf`; `allOf` (but **not** `allOf` with `$ref`); internal `$ref`/`$defs`/`definitions`; `default`; `required`; `additionalProperties`, which **must be present and** `false`; ten string formats (`date-time`, `time`, `date`, `duration`, `email`, `hostname`, `uri`, `ipv4`, `ipv6`, `uuid`); array `minItems` of **0 or 1 only**; and `pattern` with `^...$`, `*`/`+`/`?`/simple `{n,m}`, `[]`/`.`/`\d`/`\w`/`\s` and groups.

Not supported: recursion; complex enum members; external `$ref`; **all numerical constraints**; `minLength`/`maxLength`; other array constraints; `additionalProperties` set to anything but `false`; and in `pattern`, backreferences, lookaround, `\b`/`\B` and large `{n,m}`.

`oneOf` **appears in neither list.** Chapter 23's rule stands: a fact documented in neither the supported nor the unsupported list is **UNKNOWN**, and §32.18's checker raises rather than guessing.

Then the SDK layer, which is where the silence comes in. The Python, TypeScript, Ruby and PHP SDKs *"silently transform the schema"*: they strip `minimum`, `maximum`, `multipleOf`, `minLength` and `maxLength` **into the description**, add `additionalProperties: false`, filter unsupported formats, and validate the response client-side against your original schema afterwards.

Read that last clause carefully, because it is the difference between a bug and a design. The constraint is not thrown away; it is moved from **sampling time** to **after the fact**. The model is free to emit `amount_minor: -1`; the SDK's client-side validation may then object. What you have is not a guarantee, it is a late-firing check, and it fires in your process after the model has already spent the tokens.

> WARNING
>
> **The rule**
>
> Sort your constraints into **shape** and **safety**. Shape — types, enums, patterns, required fields, formats — is enforced by the grammar. Safety — ranges, lengths, quantities, limits — is not. Never rely on the grammar for safety, and validate at the call site before you execute. Chapter 34 makes that a contract.

32.5 The Three Limits Are One Budget

Chapter 23 recorded the numbers; this chapter makes the scope explicit, because the scope is what surprises people. **Combined across all strict schemas AND the JSON output schema in one request:**

Limit	Value
Strict tools	**20**
Optional parameters	**24**
Union-typed parameters	**16**

Not per tool. Not per schema. Per **request**. A team that reviews each tool in isolation will find every one of them modest and still be rejected, and §32.18 measures exactly that: nine single-purpose pull-request tools with four optional parameters each is 36 optional parameters against a cap of 24.

There are also undocumented internal limits — *"Schema is too complex for compilation"* — and a **180-second compilation timeout**, both from Chapter 23. And a subtlety worth remembering from the same chapter: 40 schemas that differ only in their `description` compile to **one** grammar, because the grammar cache is keyed on schema *structure* and the tool set, not on names or descriptions. The grammar cache and the prompt cache have different keys, and the edit that is free for one can be ruinous for the other.

32.6 Consolidation, Priced

The documented advice is direct: *"Rather than creating a separate tool for every action (`create_pr`, `review_pr`, `merge_pr`), group them into a single tool with an `action` parameter. Fewer, more capable tools reduce selection ambiguity and make your tool surface easier for Claude to navigate."*

Selection ambiguity is Chapter 33's subject. What this chapter can do is price the other half. Consolidating nine pull-request tools into one `github_pr` tool with an `action` enum, measured in §32.18:

	Nine tools	Two tools
Strict tools used	9	**2**
Optional parameters	**36 — rejected**	**8**
Accepted as a request	no	yes
Three `input_examples` each	540–1,350 tokens	**120–300**

The consolidated tool has *more* parameters than any of the nine it replaced and uses **less** of the budget, because the four optional parameters that were duplicated across nine schemas are now declared once. This is the reason consolidation is an engineering decision and not a stylistic one.

The counter-pressure is real and should be stated: an `action` enum with eight members moves a distinction out of the type system and into a runtime branch, and it makes `allowed_callers` and per-operation authorisation coarser. Chapter 34 handles the second half of that objection; the first is the ordinary trade you make whenever you replace nine functions with one dispatcher.

32.7 `input_examples`

"Examples are included in the prompt alongside your tool schema, showing Claude concrete patterns for well-formed tool calls." Chapter 20's rule that examples should be relevant, diverse and structured applies unchanged; what is new is that these examples are **schema-validated**, so a stale example is a 400 rather than a slow drift.

Three constraints:

- **Schema validation.** *"Each example must be valid according to the tool's* `input_schema`*. Invalid examples return a 400 error."* §32.18 reproduces four distinct ways to fail this offline.
- **Not supported for server tools.** They work on user-defined and Anthropic-schema client tools only.
- **Token cost.** *"~20–50 tokens for simple examples, ~100–200 tokens for complex nested objects"*, per example, on every request.

The documented priority is worth quoting because it settles an argument people have: *"Prioritize descriptions, but consider using* `input_examples` *for complex tools."* The description first; examples for nested objects, optional parameters and format-sensitive inputs. And because examples sit in the `tools` prefix, they cache — Chapter 30's first prefix level — so their per-request cost is a read rather than a write once the conversation is warm.

32.8 `tool_choice`, and What Each Option Costs

Four values:

Value	Meaning	Default when
`auto`	Claude decides whether to call a tool	`tools` are provided
`any`	Must call one of the tools, unspecified which	—
`tool`	Must call the named tool	—
`none`	May not call any tool	no `tools` are provided

Three consequences that are not obvious from the list.

Forcing suppresses the preamble. *"When you have* `tool_choice` *as* `any` *or* `tool`*, the API prefills the assistant message to force a tool to be used. This means that the models will not emit a natural language response or explanation before* `tool_use` *content blocks, even if explicitly asked to do so."* If you want both an explanation and a specific tool, the documented approach is `auto` plus an instruction in the **user** message: *"What's the weather like in London? Use the get_weather tool in your response."* Chapter 21's rule about the authority of channels is visible here — the prefill is the operator forcing a shape, and no amount of asking overrides it.

Forcing is not universally available. With **manual extended thinking** (`thinking: {"type": "enabled"}`), only `auto` and `none` are supported and `any`/`tool` return an error — Chapter 18 recorded this. **Adaptive thinking supports forced tool use**, including on models where thinking is on by default such as Opus 5. And **Claude Mythos Preview does not support forced tool use at all**: `any` or `tool` return a 400, and the documented workaround is `auto` plus prompting.

Changing it costs the message cache. Chapter 30's invalidation table: *"changes to the* `tool_choice` *parameter will invalidate cached message blocks. Tool definitions and system prompts remain cached, but message content must be reprocessed."* A router that varies `tool_choice` per request is re-prefilling its conversation every time.

The documented combination worth knowing is `tool_choice: {"type": "any"}` with `strict: true`, which *"guarantees both that one of your tools will be called AND that the tool inputs strictly follow your schema"* — subject, as ever, to §32.4's definition of "strictly".

32.9 The Array Is a Cache Prefix

Chapter 30 established that the hashed prefix is `tools` → `system` → `messages`, and that *"modifying tool definitions (names, descriptions, parameters) invalidates the entire cache"* — tools, system and messages together. `tools` is the first level, so an edit there is the most expensive edit in the API.

§32.18 prices it: six turns with a stable 3,000-token tools array cost **28,200** base-input units with one write and five reads; the same six turns with the tools array rebuilt each turn cost **97,200**, with six writes and zero reads. **3.45 times the input line**, for a change that is usually accidental.

The accidents are well catalogued. A tool list assembled from a `set` or a dictionary iterated in a language that does not guarantee order. A description regenerated with a timestamp in it. A schema serialised by a language that randomises JSON key order — Chapter 30 named Swift and Go specifically. A per-user tool list that grows as permissions are evaluated.

The remedies are three: fix the order and sort the keys; place `cache_control` on the **last** tool so the whole prefix caches, and on the `mcp_toolset` entry itself when the tools come from a toolset whose internal order you do not control; and use `defer_loading` with the tool search tool when the set genuinely must grow, because a discovered tool arrives as a `tool_reference` block in the message history and *"the prefix is untouched, so prompt caching is preserved."*

32.10 Property Ordering

One small documented behaviour catches people building parsers and logs. Chapter 23 recorded it: **required properties are emitted first, then optional, each in schema order.** The order you wrote the properties in is not the order they arrive.

§32.18 demonstrates it on the transfer schema, whose source order begins `notify, reference, account_id, …` and whose emitted order begins `account_id, amount_minor, currency, …`. Anything that compares serialised tool inputs — a cache key, a deduplication hash, a golden-file test — must normalise before comparing, or it will report differences that do not exist.

32.11 Visual Analogies

VISUAL ANALOGY

The form and the clerk

A schema under `strict: true` is a paper form with boxes of fixed shapes. You cannot write a word in the number box or a date in the tick box — the form makes it physically impossible. But nothing about the form stops you writing a negative amount in the amount box, because the box is shaped for digits and does not know what a bank is. The clerk behind the counter checks that.

Where it breaks: a real form can be printed with a smaller box to limit the length. Here the length limit is not printed on the form at all — it is written in the covering letter, addressed to the person filling it in.

VISUAL ANALOGY

One budget, many pockets

The three strict limits are a single household budget rather than nine separate allowances. Every tool takes from the same pot, and so does the JSON output schema, so a tool that looks affordable on its own may still be unaffordable.

Where it breaks: a household budget is spent as you go, and you notice when it is empty. This one is checked all at once when the request arrives, so you find out about the ninth tool and not the first.

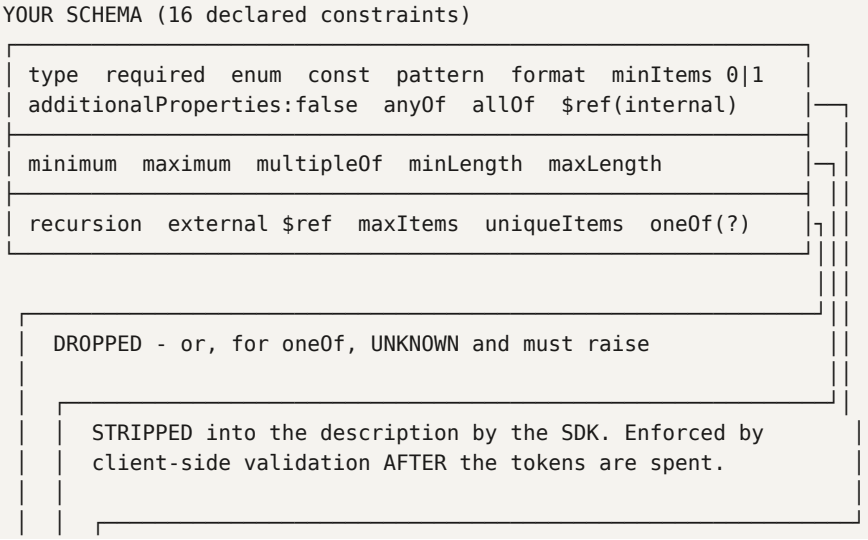
VISUAL ANALOGY

The trained-in signature

Chapter 31 noted that Anthropic-schema tools are called more reliably because the model has seen those exact signatures many times. A schema is a signature: familiar shapes are called correctly more often than clever ones, in the same way an experienced engineer picks up an unfamiliar library faster when its API looks like the ones they know.

Where it breaks: an engineer can read your source when the signature confuses them. The model cannot — *"Claude never sees your implementation; it only sees the schema you provided and the result you returned."*

32.12 Figures



```
   |  |  |  ENFORCED by the grammar at sampling time.
   ▼  ▼  ▼
   0  3  13   ->  enforcement coverage 81.2%
   Everything in the enforced row is SHAPE.
   Everything in the stripped row is SAFETY.
```

```
  tools[]                          strict   optional   union
  ├─ create_pr    (4 optional)       1         4         0
  ├─ review_pr    (4 optional)       1         4         0
  ├─ merge_pr     (4 optional)       1         4         0
  ├─ close_pr     (4 optional)       1         4         0
  ├─ list_prs     (4 optional)       1         4         0
  ├─ comment_pr   (4 optional)       1         4         0
  ├─ label_pr     (4 optional)       1         4         0
  ├─ assign_pr    (4 optional)       1         4         0
  └─ transfer_funds                  1         4         0
  output_config.format               -        +n         -
                                    ────      ────      ────
                                     9        36         0
                                    /20      /24        /16
                                              ^^ 400
  after consolidation:
  ├─ github_pr  (action enum, 4 optional)   1     4     0
  └─ banking_transfer                       1     4     0
                                           ────  ────  ────
                                            2     8     0   accepted
```

32.13 Examples

Example 1 — the three required fields, plus namespacing.

```
{"name": "github_pr",                       # ^[a-zA-Z0-9_-]{1,64}$
 "description": "Create, review, merge, close, list, comment on, "
                "label or assign a pull request. Returns the PR's "
                "slug and state, not its full diff.",
 "input_schema": {"type": "object", "additionalProperties": False,
                  "properties": {...}, "required": ["action", "id"]}}
```

Example 2 — a safety constraint the grammar cannot keep.

```
"amount_minor": {"type": "integer", "minimum": 1,
                 "maximum": 1_000_000,      # STRIPPED into description
                 "description": "Amount in minor units."}
```

Example 3 — the same constraint, kept where it can be enforced.

```
def transfer(**kw):
    amount = kw["amount_minor"]
    if not 1 <= amount <= 1_000_000:        # the grammar will not do
        return error("amount_minor out of range")   # this for you
```

Example 4 — examples that must validate.

```
"input_examples": [
```

```
    {"account_id": "6f1e...", "amount_minor": 2500, "currency": "GBP"},
    {"account_id": "6f1e...", "amount_minor": 100, "currency": "USD",
     "reference": "INV-2026-0041"},        # optional field shown
]
```

Example 5 — forcing a tool, and the two places it is unavailable.

```
tool_choice={"type": "any"}                # +120 tokens on Opus 5
# 400 on claude-mythos-preview.
# 400 with thinking={"type": "enabled"}; adaptive thinking is fine.
# Suppresses any natural-language preamble.
```

Example 6 — keeping the prefix stable and cached.

```
tools = sorted(TOOLS, key=lambda t: t["name"])       # fixed order
body = json.dumps(tools, sort_keys=True)             # fixed key order
tools[-1]["cache_control"] = {"type": "ephemeral", "ttl": "1h"}
```

32.17 Code Walkthrough: Counting What Is Enforced

`enforce.py` is the chapter's argument expressed as four lists. It walks a schema and puts every constraint it finds into exactly one of them.

```
    for kw in STRIPPED:
        if kw in node:
            declared.append((path, kw))
            stripped.append((path, kw))
    if "minItems" in node:
        declared.append((path, "minItems"))
        if node["minItems"] in (0, 1):
            enforced.append((path, "minItems"))
        else:
            dropped.append((path, "minItems"))
```

`minItems` gets its own branch because it is the one keyword whose fate depends on its **value** rather than its name: 0 and 1 are supported, everything else is not. A checker that classifies by keyword alone would report `minItems: 3` as enforced, which is exactly the kind of quiet wrong answer this book keeps finding.

The ratio at the end is the number the chapter publishes:

```
        "coverage": len(enforced) / len(declared) if declared else 1.0,
```

It is deliberately a ratio of **counts**, not of importance, and the chapter says so: 81.2 per cent sounds reassuring until you notice that all three of the missing constraints are the ones about money. A metric that treats `type` and `maximum` as equally weighted is the right metric for an audit and the wrong one for a decision, and §32.18 reports both the number and the list.

`tooldef.py` adds the tool-specific rules, and one of its functions exists purely to avoid overstating a failure:

```
def _sdk_handles(finding):
    return any(kw in finding.message or finding.path.endswith("." + kw)
```

```
                for kw in S.SDK_STRIPPED)
```

Chapter 23's analyser classifies `minimum` as an **error**, which is correct for an output schema written against the raw API. In a tool definition sent through an official SDK it is not an error — the request succeeds and the constraint is silently relocated. Reporting it as a 400 would be a false alarm, and false alarms are how a preflight gets switched off. So it is downgraded, counted, and published as a coverage figure instead.

`budget` is four lines and the whole point is its loop bound:

```
    for t in tools:
        rep = check_tool(t)
        if t.strict:
            b.strict_tools += 1
            b.optional_params += rep.optional_params
            b.union_params += rep.union_params
```

It iterates the **request**, not a tool. Every published limit in this area is a request-level limit, and a per-tool checker will pass a toolset the API rejects.

32.18 Mini Project: A Tool-Definition Preflight and a Budget

Folder structure.

```
ch32/
  tooldef.py     names, examples, the three limits, tool_choice rules
  enforce.py     declared vs enforced vs stripped vs dropped
  catalogue.py   nine single-purpose tools and two consolidated ones
  report.py      every published figure
  test_ch32.py   29 tests
  schema.py      copied from ch23/
  cacheplan.py   copied from ch30/
```

Result 1 — sixteen declared, thirteen enforced, three stripped

```
declared constraints             16
enforced by the grammar          13
stripped into the description     3
dropped entirely                  0
coverage                      81.2%
   STRIPPED  $.reference            maxLength
   STRIPPED  $.amount_minor         minimum
   STRIPPED  $.amount_minor         maximum
emitted property order: account_id, amount_minor, currency, notify,
                        reference, scheduled_for, idempotency_key
source property order:  notify, reference, account_id, scheduled_for,
                        amount_minor, idempotency_key, currency
```

Thirteen of sixteen is a comfortable-sounding number attached to an uncomfortable fact. What survives is `type`, `required`, `enum`, `const`, `pattern` and `format` — the **shape** of a transfer instruction. What does not survive is the minimum amount, the maximum amount and the maximum length of the payment reference — the **safety** of one. `strict: true` on this tool guarantees that the model asks for a

well-formed transfer and guarantees nothing at all about whether the amount is one a bank should move.

The property-ordering lines are the same measurement in miniature: seven properties, seven in both lists, and a different order, because required properties are emitted first regardless of how you wrote them.

Result 2 — nine reasonable tools, one rejected request

```
nine single-purpose tools  strict= 9 optional=36 union= 0
                           over=[('optional parameters', 36, 24)]
two consolidated tools     strict= 2 optional= 8 union= 0  over=no
consolidated + a JSON output schema: optional=9
nine tools REJECTED: 36 optional parameters exceeds the documented
  limit of 24, counted across the whole request
two tools accepted: True
```

Not one of the nine tools is unreasonable. Each has one required parameter and four optional ones, which is a modest schema by any standard, and each would pass a review conducted tool by tool. The request fails because 9 × 4 = 36 and the cap is 24.

The third line is the one that catches people twice: adding a `output_config.format` JSON schema with a single optional property takes the consolidated set from 8 to **9**, because the output schema draws on the same budget as the tools. A team that has carefully sized its toolset can be pushed over by a feature added somewhere else in the request entirely.

Result 3 — twelve rejections and two UNKNOWNs, offline

```
  name with a space                              rejected
  name over 64 chars                             rejected
  empty name                                     rejected
  duplicate tool names                           rejected
  input_example missing a required property      rejected
  input_example with an undeclared property      rejected
  input_example of the wrong type                rejected
  input_example outside an enum                  rejected
  input_examples on a server tool                rejected
  forced tool use on Mythos Preview              rejected
  forced tool use with manual extended thinking  rejected
  over the optional-parameter budget             rejected
documented rejections reproduced offline: 12
  UNKNOWN  oneOf in a strict tool schema -> raised
  UNKNOWN  an undocumented tool_choice value -> raised
facts recorded UNKNOWN (each raises): 2
```

Four of the twelve are `input_examples` failures, which is worth dwelling on because the feature is optional and therefore easy to treat as decoration. It is not: an example that drifts out of line with its schema is a **400 on every request**, so an example is a piece of code with a test, not a comment.

The two UNKNOWNs behave the way this book has behaved since Chapter 14. `oneOf` is in neither the supported nor the unsupported list, so the checker raises `UnknownSchemaFeature` rather than guessing which way it will go. An undocumented `tool_choice` value raises `Unverified` rather than being passed through on the theory that the server will sort it out.

Result 4 — what a tools array costs before anything is called

```
  model             tool_choice auto/none   any/tool
  claude-haiku-4-5               496            588  (+92)
  claude-opus-5                  286            406  (+120)
  claude-sonnet-5                354            474  (+120)
input_examples: (20, 50) tokens simple, (100, 200) complex, per example
three simple examples on each of nine tools: 540-1350 tokens
the same on two consolidated tools:          120-300 tokens
```

Forcing a tool is **120 tokens per request** on Opus 5 and Sonnet 5. On a workload doing a million requests a month that is 120 million input tokens, which at Opus 5's rate is $600 a month for a parameter. It caches, so the real figure is a tenth of that once the prefix is warm — which is the point of the next result.

Consolidation shows up again here: the same three examples per tool cost between four and five times less on two tools than on nine, and examples sit in the cacheable prefix, so the saving is a one-off write rather than a per-request charge.

Result 5 — the most expensive edit in the API

```
stable tools array     billed units = 28200.0  writes=1 reads=5
tools edited each turn billed units = 97200.0  writes=6 reads=0
editing the tools array costs 3.45x the input line
```

Six turns, one 3,000-token tools array, one 9,000-token system prompt. Held stable, the conversation writes the cache once and reads it five times. Rebuilt each turn — a regenerated description, an unsorted dictionary, a per-user tool list — it writes six times, reads never, and costs **3.45 times** as much. `tools` is the first level of the prefix hierarchy, so an edit there invalidates the system prompt and the entire message history along with it. This is Chapter 30's rule with a price attached.

What the project establishes

Four numbers. **81.2 per cent** enforcement coverage on a careful schema, with every missing constraint a safety one. **36 against 24**, the budget nine modest tools spend. **12** documented rejections reproducible with no key, and **2** facts that must raise. And **3.45×**, the cost of an unstable tools array.

32.19 Hands-on Labs

Lab 1 — Audit your own schemas (success: a coverage figure and a list). Run `enforce.audit` over every tool you ship and list the constraints that are stripped or dropped.

Lab 2 — Compute your request budget (success: three numbers against 20/24/16). Count strict tools, optional parameters and union-typed parameters across the whole request, including any `output_config.format`.

Lab 3 — Consolidate (success: a lower budget with no loss of capability). Group related operations behind an `action` enum and re-measure.

Lab 4 — Break `input_examples` on purpose (success: four distinct rejections). Missing required, undeclared property, wrong type, outside an enum.

Lab 5 — Prove your tools array is byte-stable (success: one hash across ten turns). Serialise with sorted keys and a fixed order, hash it every turn, assert it never changes.

32.20 Lab Solutions

Lab 1. Classify by *value* as well as by keyword: `minItems` of 0 or 1 is enforced and anything else is dropped. Then re-sort the stripped list by whether the constraint protects a safety property; the count matters less than that partition.

Lab 2. Iterate the request, not the tool. Remember the output schema. If you are near the cap, remember that `strict: false` on a tool removes it from the strict-tool count entirely — at the cost of its input guarantees, which §32.4 has already shown are narrower than they look.

Lab 3. The consolidated tool needs a longer description, not a shorter one: it now has to explain when each `action` applies. Chapter 33 covers how to write it. Do not consolidate operations with different authorisation requirements without reading Chapter 34 first.

Lab 4. Validation is recursive; nested objects and array items fail the same four ways. The most common real failure is the fourth — an enum gains a member in the schema and an example is left behind, or loses one and an example is orphaned.

Lab 5. `json.dumps(tools, sort_keys=True)` fixes key order within each object; you must fix the order of the array yourself. If any tool description contains a formatted date, a version string or a user's name, the hash will move and so will your bill.

32.21 Best Practices

BEST PRACTICE

1. **Partition every constraint into shape and safety**, and never rely on the grammar for safety.
2. **Validate at the call site** before executing, against your original schema.
3. **Count the three limits across the whole request**, including `output_config.format`.
4. **Consolidate related operations** behind an `action` enum; measure the budget before and after.
5. **Namespace tool names by service** — `github_list_prs`, not `list_prs`.
6. **Write at least three or four sentences of description**, covering what the tool does, when it should and should not be used, what each parameter means, and what it does **not** return.
7. **Set** `additionalProperties: false` **explicitly** on every object; it is required and not defaulted.
8. **Treat** `input_examples` **as code**: they are schema-validated and a stale one is a 400.
9. **Keep the tools array byte-stable** — fixed order, sorted keys, no interpolated values.
10. **Place** `cache_control` **on the last tool**, or on the `mcp_toolset` entry when you do not control the order.
11. **Use** `defer_loading` **and tool search** when the set must grow mid-conversation.
12. **Do not vary** `tool_choice` **per request** unless you accept re-prefilling the messages.
13. **Normalise before comparing tool inputs**, because required properties are emitted first.
14. **Raise on** `oneOf` and on any keyword documented in neither list.

32.22 Common Mistakes

COMMON MISTAKE

Trusting `strict: true` with a safety property

Symptom: a tool call arrives with a negative amount, an over-long string or an out-of-range quantity, on a schema that declares limits for all three. **Cause:** numerical constraints and `minLength`/`maxLength` are outside the supported subset and are stripped into the description by the SDK. **Fix:** validate ranges and lengths at the call site; treat the schema's copy as documentation for the model. **Detection:** an audit that counts declared against enforced — 81.2 per cent on the chapter's schema, with all three misses on the safety side.

COMMON MISTAKE

Reviewing tools one at a time

Symptom: a 400 about optional parameters after adding a ninth modest tool. **Cause:** the limits are per request, not per tool, and the JSON output schema shares them. **Fix:** a budget check across the whole request in CI. **Detection:** sum `optional_params` across every strict schema plus `output_config.format`.

COMMON MISTAKE

A tools array rebuilt every turn

Symptom: `cache_read_input_tokens` permanently zero; input costs roughly 3.45× the estimate. **Cause:** unstable ordering, randomised JSON key order, a description containing a timestamp or a per-user list. **Fix:** fixed order, `sort_keys=True`, no interpolation; use `defer_loading` to grow the set. **Detection:** hash the serialised array each turn and assert it is constant.

COMMON MISTAKE

A stale `input_example`

Symptom: a 400 on every request after an unrelated schema change. **Cause:** an example that no longer validates — usually an enum member added or removed. **Fix:** validate examples against the schema in your test suite, exactly as the API does. **Detection:** the four checks in §32.18's Result 3.

COMMON MISTAKE

Forcing a tool where forcing is unavailable

Symptom: a 400 on `tool_choice: {"type": "any"}` that works fine on another model or another request. **Cause:** Claude Mythos Preview does not support forced tool use; manual extended thinking supports only `auto` and `none`. **Fix:** `auto` plus an explicit instruction in the user message; adaptive thinking supports forcing. **Detection:** a preflight keyed on model and thinking mode.

COMMON MISTAKE

Expecting a natural-language preamble under `any` or `tool`

Symptom: the model stops explaining itself, and prompting for an explanation has no effect. **Cause:** forcing prefills the assistant message, so no text can precede the `tool_use` blocks. **Fix:** use `auto` and ask for the tool by name in the user message. **Detection:** assert on the presence of a `text` block before the first `tool_use`.

COMMON MISTAKE

Assuming property order

Symptom: golden-file tests or deduplication hashes report differences that are not real. **Cause:** required properties are emitted first, then optional, each in schema order. **Fix:** normalise key order before comparing or hashing. **Detection:** compare the emitted order against your source order once; if they differ, you have the bug.

32.23 Performance Considerations

The tools array is the first level of the cache prefix and usually the most stable thing in the request, which makes it the best thing in the request to cache and the worst thing to disturb. Place the breakpoint on the last tool; §32.18 measures the cost of not doing so at 3.45×.

Description length is a real cost with a real return. The documentation asks for *"at least 3–4 sentences for each tool description, more if the tool is complex"*, and Chapter 33 will show the selection accuracy that buys. Because descriptions live in the cached prefix, their marginal cost after the first request is 0.1× — which is the strongest argument in the book for writing them properly. A 400-token description read a thousand times costs less than a 40-token description that causes one wrong tool call.

Two limits from Chapter 23 bite at scale rather than in development: an undocumented internal grammar-size limit returning *"Schema is too complex for compilation"*, and a **180-second compilation timeout**. Both argue for fewer, flatter schemas. And the grammar cache holds for 24 hours from last use and is invalidated by schema **structure** or by the **tool set** — but not by `name` or `description` changes, so editing prose is free for the grammar cache and expensive for the prompt cache. The two caches have different keys and opposite sensitivities, which is worth writing on a wall.

32.24 Security Considerations

A schema is not an authorisation boundary. It constrains what a request can look like, not who may make it. `allowed_callers` and your own call-site checks are the boundary; Chapter 34 builds them. The tool named `delete_records` with a `confirm: true` parameter has not been made safe by adding the parameter — it has delegated the decision to the model.

Safety constraints must live where they are enforced. §32.18's measurement is a security finding as much as a cost one: the amount limits on a payments tool are not enforced at sampling time, and a system that treated `strict: true` as a control would have a control that does not exist.

Do not put secrets in a schema. Chapter 23 recorded that the schema is cached separately from message content and *"does not receive the same PHI protections"* — no PHI in property names, enum values, `const` values or `pattern` regexes. The same reasoning covers internal hostnames, table names and identifiers.

Every tool you send is a capability you have granted for that request. Least privilege applies to the array itself, and `defer_loading` lets you keep the granted set small without losing reach.

Descriptions and enum values are read by the model and by anything that logs the request. Chapter 33 will note that a description is a prompt; here it is enough to say that it is also an artefact you are shipping to a third party's servers on every call.

32.25 Real Production Examples

A payments API. Shape in the schema, safety at the call site, and a unit test asserting that the audit's stripped list is exactly the set the call site re-checks. That test is the cheapest way to keep the two in step.

A GitHub integration. Nine operations consolidated to one `github_pr` tool with an `action` enum, `github_` namespacing throughout, and a budget check in CI that fails the build at 20 optional parameters rather than 24, leaving room for an output schema.

A support assistant with per-user tools. The tool list varied by the caller's permissions and never cached. Rewritten to send the full list with `allowed_callers` and call-site authorisation, restoring a stable prefix and cutting the input line by roughly the 3.45× $32.18 measures.

A multi-tenant platform pulling tools from MCP servers. Schemas it does not own, so the preflight runs on every deploy: budget, supported subset, `oneOf` raising, examples validated. Part 7 takes up what changes when the schema is someone else's.

32.26 Certification Notes

Concept	Exam domain	Weight
Tool definition fields and the name regex	D2 Tool Design & MCP Integration	18%
`strict: true` and the supported subset	D4 Prompt Engineering & Structured Output	20%
The three limits, counted per request	D2 Tool Design & MCP Integration	18%
`input_examples` rules and cost	D2 Tool Design & MCP Integration	18%
`tool_choice` and its compatibility matrix	D1 Agentic Architecture & Orchestration	27%
Tools as the first cache prefix level	D5 Context Management & Reliability	15%
Validating safety constraints at the call site	D5 Context Management & Reliability	15%

CERTIFICATION NOTE

Exam Tips

- The name regex is `^[a-zA-Z0-9_-]{1,64}$` — no dots, no slashes, no spaces.
- `additionalProperties` **must be present and** `false`**.** Absent is not the same as false.
- **All numerical constraints and** `minLength/maxLength` **are unsupported** and stripped into the description by the SDKs.
- The limits **20 / 24 / 16** are combined **across the whole request**, including the JSON output schema.
- `oneOf` **is in neither list** — the correct answer is UNKNOWN.
- `input_examples` must validate, are **not supported on server tools**, and cost 20–50 or 100–200 tokens each.
- `any` and `tool` **prefill the assistant message**, so no natural-language preamble appears.
- Forced tool use is **rejected on Mythos Preview** and with **manual extended thinking**; adaptive thinking supports it.
- Changing `tool_choice` invalidates the **messages** cache; changing `tools` invalidates **everything**.
- **Required properties are emitted first**, then optional, each in schema order.

32.27 Twenty Practice Questions

1. What are the three required fields on a tool definition?
2. Give the tool name regex and two names it rejects.
3. What does `strict: true` guarantee about the tool `name`?
4. Which of `minimum`, `pattern`, `maxLength`, `enum` survive to sampling time?
5. What do the Python and TypeScript SDKs do with a `maximum` keyword?
6. Scope the limits 20 / 24 / 16.
7. Eight strict tools with three optional parameters each, plus a JSON output schema with two optional properties. Is the request accepted?
8. What happens if an `input_example` does not validate?
9. Can you use `input_examples` on `web_search`?
10. Name the four `tool_choice` values and their defaults.
11. Why does no explanatory text appear before a forced tool call?
12. On which model does forced tool use return a 400?
13. Which thinking mode is incompatible with forced tool use?
14. What does changing `tool_choice` invalidate? What does changing `tools` invalidate?
15. Where should `cache_control` go in a `tools` array, and where for an `mcp_toolset`?
16. What is the emitted order of properties?
17. Why is `oneOf` not simply treated as unsupported?
18. What does `defer_loading` preserve?
19. Give the documented reason to consolidate `create_pr`, `review_pr` and `merge_pr`.

20. Your schema declares 16 constraints and 13 are enforced. Why is 81.2 per cent the wrong thing to report on its own?

32.28 Answers

1. `name`, `description`, `input_schema`. **Wrong distractors:** `input_examples` and `strict` are optional; `parameters` is another vendor's field name.
2. `^[a-zA-Z0-9_-]{1,64}$`. It rejects `get weather` (space) and `github.list_prs` (dot); also anything empty or over 64 characters.
3. That the emitted `name` is one of the tools you declared — a genuinely useful guarantee, and separate from the input constraint.
4. `pattern` **and** `enum` **survive.** `minimum` and `maxLength` do not.
5. They **strip it into the description**, add `additionalProperties: false`, filter unsupported formats, and validate the response client-side against your original schema afterwards.
6. Combined across **all strict schemas and the JSON output schema in one request** — not per tool.
7. 8 × 3 = 24, plus 2 from the output schema, is **26 optional parameters against a cap of 24**: rejected. The trap is counting only the tools.
8. A **400 error**, on every request carrying it.
9. **No.** `input_examples` work on user-defined and Anthropic-schema client tools only.
10. `auto` (default when tools are provided), `any`, `tool`, `none` (default when no tools are provided).
11. Under `any` or `tool` *"the API prefills the assistant message to force a tool to be used"*, and no text may precede the `tool_use` blocks *"even if explicitly asked"*.
12. **Claude Mythos Preview.** Use `auto` plus prompting.
13. **Manual extended thinking** (`thinking: {"type": "enabled"}`) supports only `auto` and `none`. Adaptive thinking supports forcing.
14. `tool_choice` invalidates the **messages** cache, leaving tools and system cached. `tools` invalidates **tools, system and messages** — the whole prefix.
15. On the **last tool** in the array; for an `mcp_toolset`, on the **toolset entry itself**, and the API applies it to the final expanded tool.
16. **Required properties first, then optional, each in schema order** — not the order you wrote them.
17. Because it appears in **neither** the supported nor the unsupported list. Treating an absence of documentation as a documented negative is a guess; UNKNOWN must raise.
18. The cached prefix. A deferred tool is not in the prefix, and when tool search discovers it the definition arrives as a `tool_reference` block **in the message history**.
19. *"Fewer, more capable tools reduce selection ambiguity and make your tool surface easier for Claude to navigate."* Measured here: 9 tools and 36 optional parameters become 2 and 8.
20. Because the metric weights every constraint equally and the three that are missing are the ones about money. Report the **list** alongside the ratio.

32.29 Ten Practical Exercises

1. Audit every production schema and publish the stripped list.
2. Add a CI check for the three request-level limits with a margin below the cap.
3. Write a test asserting your call-site validation covers exactly the stripped set.
4. Consolidate one family of tools and measure the budget change.
5. Add `input_examples` to your most format-sensitive tool and validate them in tests.
6. Hash your serialised `tools` array on every request and alert on change.
7. Move `cache_control` to the last tool and measure `cache_read_input_tokens`.
8. Find every tool whose description is under three sentences.
9. Grep your schemas for identifiers, hostnames or anything resembling PHI.
10. Check whether any code compares serialised tool inputs without normalising key order.

32.30 Five Challenge Labs

1. **The schema linter.** Fail a build on: a missing `additionalProperties: false`, an unsupported keyword, `oneOf`, a budget over 20/24/16, an invalid `input_example`, or a description under three sentences.
2. **The consolidation optimiser.** Given a set of operations and their parameters, propose a grouping minimising total optional parameters subject to keeping authorisation boundaries intact.
3. **The drift detector.** Compare your call-site validation against `enforce.audit`'s stripped list and fail when they diverge.
4. **The prefix stabiliser.** Serialise a tools array deterministically across two languages and prove the bytes match.
5. **The cost model.** Extend Chapter 30's cache planner to price a toolset: definitions, examples, the tool-use system prompt and the `tool_choice` premium, over a realistic conversation length.

32.31 Ten Interview Questions

1. What does `strict: true` actually guarantee?
2. Where would you enforce a maximum transfer amount, and why not in the schema?
3. How do the strict limits scope, and why does that surprise people?
4. When would you consolidate tools, and what do you give up?
5. What is the cost of an unstable `tools` array?
6. Why is `oneOf` interesting?
7. What does forcing a tool cost, in three different senses?
8. How do the grammar cache and the prompt cache differ?
9. What is in a tool description, and why does length not cost what you think?
10. A tool call arrives with an out-of-range value on a strict schema. Walk me through the diagnosis.

32.32 Summary

A tool definition is three required fields and a great deal of consequence. The `name` is matched against a regex and is also, as Chapter 33 will show, the strongest single signal in tool selection. The `description` is prose addressed to a language model and belongs in the cached prefix, where its marginal cost after the first request is a tenth of an input token and its return is a call that goes to the right place. The `input_schema` is the field this chapter is about, and the thing to understand about it is the gap between what it declares and what any of it enforces.

Under `strict: true`, constrained decoding restricts the token distribution so that the emitted `input` conforms to the **supported subset** of your schema, and guarantees that the emitted tool `name` is one you declared. The supported subset covers types, `required`, `enum`, `const`, `pattern`, ten string formats, `minItems` of exactly 0 or 1, and `additionalProperties` which must be present and `false`. It does not cover any numerical constraint, or `minLength` and `maxLength`, or the other array constraints, and the official SDKs strip those keywords into the description rather than rejecting them — so a schema that declares a minimum, a maximum and a length limit compiles cleanly and enforces none of the three. On the payments schema audited here, thirteen of sixteen declared constraints survive to sampling time, and all three that do not are the constraints protecting money. Shape is enforced; safety is not; and the design rule that follows is to sort every constraint into one of those two piles and never leave anything from the second pile in a place the grammar cannot reach.

The second surprise is arithmetical rather than semantic. The three strict limits — 20 tools, 24 optional parameters, 16 union-typed parameters — are combined across every strict schema and the JSON output schema in a single request. Nine single-purpose tools with four optional parameters each is thirty-six against a cap of twenty-four, and every one of the nine passes a review conducted in isolation. The documented remedy is consolidation behind an `action` parameter, and consolidation turns out to pay twice: nine tools and thirty-six optional parameters become two and eight, and the same three worked examples per tool cost between four and five times less to carry. Adding a JSON output schema with one optional property then pushes the count from eight to nine, which is the reminder that this is a request-level budget and not a toolset-level one.

Everything else in the chapter follows from the tools array being the first level of Chapter 30's cache prefix. Editing it invalidates the system prompt and the entire message history along with it, at a measured cost of 3.45 times the input line over six turns; changing `tool_choice` is cheaper but not free, invalidating the messages cache while leaving tools and system intact; and forcing a tool costs 120 tokens on Opus 5 and Sonnet 5 before anything happens, suppresses any natural-language preamble, and is unavailable on Mythos Preview and under manual extended thinking. Chapter 33 takes up the words. Chapter 34 takes up the validation this chapter has repeatedly deferred to the call site, and gives it a contract.

KEY TAKEAWAYS

1. **Three required fields:** `name` matching `^[a-zA-Z0-9_-]{1,64}$`, `description`, and `input_schema`.
2. `strict: true` **guarantees a valid tool name and conformance to the supported subset** — not to your schema.
3. **Shape is enforced; safety is not.** Numerical constraints and `minLength`/`maxLength` are stripped into the description by the SDKs; 13 of 16 constraints survived on the audited schema, and all 3 misses were safety ones.
4. **Validate ranges and lengths at the call site**, against your original schema.
5. **20 / 24 / 16 are one budget for the whole request**, output schema included: nine modest tools spent 36 of 24.
6. **Consolidation buys budget and tokens**, taking 9 tools and 36 optional parameters to 2 and 8.
7. `input_examples` **are schema-validated code**, unavailable on server tools, and cost 20–200 tokens each.
8. `any` **and** `tool` **prefill the assistant turn**, cost 120 extra tokens, and are rejected on Mythos Preview and under manual extended thinking.
9. `tools` **is the first cache level.** An unstable array costs 3.45× the input line; `tool_choice` changes cost the messages cache.
10. **A keyword in neither list is UNKNOWN and must raise** — `oneOf` above all.

32.33 Glossary

Term	Definition
`action` parameter	An enum consolidating several operations into one tool
`additionalProperties`	Must be present and `false` under strict mode; absent is not the same as false
`allowed_callers`	An optional tool property restricting who may invoke it (Chapter 35)
`defer_loading`	Keeps a tool out of the cached prefix until tool search discovers it
Enforcement coverage	Declared constraints divided by constraints enforced at sampling time
Grammar cache	The 24-hour cache of a compiled schema, keyed on structure and tool set, not on names or descriptions
`input_examples`	Schema-validated example inputs; a 400 if invalid; not supported on server tools
`input_schema`	The JSON Schema describing a tool's parameters
Namespacing	Prefixing tool names by service, for example `github_list_prs`
Optional parameter budget	The 24 non-required parameters allowed across all strict schemas in one request
Property ordering	Required properties emitted first, then optional, each in schema order
Strict tool budget	The 20 strict tools allowed in one request

Term	Definition
`strict: true`	Constrained decoding on a tool's input plus a guaranteed valid tool name
Supported subset	The JSON Schema keywords constrained decoding can compile
Tool-use system prompt	The system prompt the API builds from your tools; 286–588 tokens depending on model and `tool_choice`
Union parameter budget	The 16 union-typed parameters allowed across one request

32.34 Further Reading

Primary sources, verified 2 August 2026 at `platform.claude.com/docs/en`:

- `agents-and-tools/tool-use/define-tools` — the fields, the name regex, the tool-use system prompt, `input_examples`, `tool_choice` and the forced-tool-use restrictions.
- `agents-and-tools/tool-use/strict-tool-use` and `build-with-claude/structured-outputs` — the supported subset, the three limits and the SDK transformations, verified in Chapter 23.
- `agents-and-tools/tool-use/tool-reference` — the full set of optional tool properties including `cache_control`, `strict`, `defer_loading` and `allowed_callers`.
- `agents-and-tools/tool-use/tool-use-with-prompt-caching` — breakpoint placement and `defer_loading`, verified in Chapter 30.
- Anthropic Engineering, *Writing tools for agents* — the essay the documentation points to for consolidation, naming and response shaping.

Where to go next in this book. Chapter 33 covers the words: names, descriptions, affordances, and what they do to selection accuracy. **Chapter 34** builds the error contract and the call-site validation this chapter has deferred to five times. **Chapter 35** takes a toolset up to many tools across many turns, where the budget and the cache prefix meet orchestration. **Chapter 36** opens Part 7 with MCP, and with schemas you did not write. **Chapter 58** covers what an attacker can do with a schema you did.